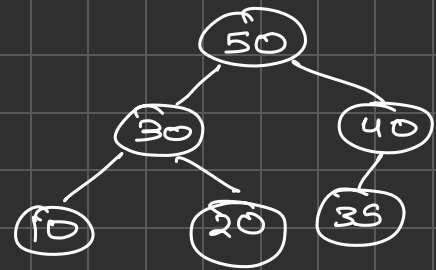


HEAPS, GRAPHS & TREES

⇒ Heap:

- Top element: $O(1)$ (min/max)
- Insert/Delete: $O(\log(n))$
- Build heap: $O(n)$
- Search: $O(n)$



- Conceptually represented as tree's.
- Stored as array

0 1 2 3 4 5
[50, 30, 40, 10, 20, 15]

- If an element is at index 'i'
 - Parent → $(i-1)/2$
 - Left child → $2i+1$
 - Right child → $2i+2$

∴ It's a complete binary tree (i.e., all levels are full, except the last level)

GRAPHS

⇒ BFS, DFS, Dijkstra's, ...

⇒ For BFS & DFS:

$$T.C = O(V+E)$$

$$S.C = O(V)$$

⇒ For Dijkstra's (doesn't work with graph's with -ve weight)

$$T.C = O((V+E) \log V)$$

$$S.C = O(V)$$

PROBLEM-1

973. K Closest Points to Origin

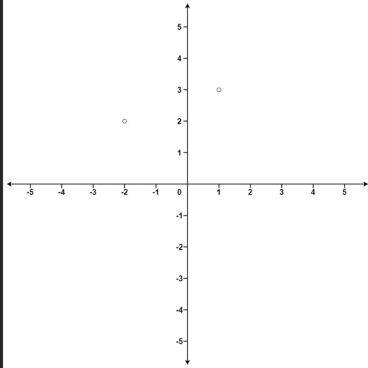
Medium Topics Companies

Given an array of points where points[i] = [x_i, y_i] represents a point on the X-Y plane and an integer k, return the k closest points to the origin (0, 0).

The distance between two points on the X-Y plane is the Euclidean distance (i.e., $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$).

You may return the answer in any order. The answer is guaranteed to be unique (except for the order that it is in).

Example 1:



Input: points = [[1,3], [-2,2]], k = 1
Output: [[-2,2]]

points = [[1, 3], [-2, 2]]
k = 1
origin = (0, 0)

Naive / Brute:

$$\sqrt{(1-0)^2 + (3-0)^2} = \sqrt{10}$$

$$\sqrt{(-2-0)^2 + (2-0)^2} = \sqrt{8}$$

$$\sqrt{x^2 + y^2}$$

Brute
T.C = $O(n) + O(\sqrt{n \log n})$
T.C = $O(n) + O(n)$

Optimal: Heap: max_heap

points = [[1, 3], [-2, 2]]
k = 1

$$1^2 + 3^2 = 10$$

$$(-2)^2 + 2^2 = 8$$

k = 1

heap-size > k

$$\begin{aligned} \text{dist} &= \sqrt{(x-0)^2 + (y-0)^2} \\ &= \sqrt{x^2 + y^2} \\ &= \underline{x^2 + y^2} \end{aligned}$$

k = 1

heappop()
↳ max element from heap

↓
(8, (-2, 2))
~~(10, (1, 3))~~

points = [[3, 3], [5, -1], [-2, 4]], k = 2

$$i=0, 3^2 + 3^2 = 18$$

$$i=1, 5^2 + (-1)^2 = 26$$

$$i=2, (-2)^2 + 4^2 = 20$$

heap-size > k

↓
(20, (-2, 4))
~~(26, (5, -1))~~
(18, (3, 3))

$$TC = O(n (O(\log(k)) + O(\log(k))))$$

$$SC = O(n)$$

PROBLEM - 2

DFS, BFS

200. Number of Islands

Solved

Medium Topics Companies

Given an $m \times n$ 2D binary grid `grid` which represents a map of '1's (land) and '0's (water), return the number of islands.

An **island** is surrounded by water and is formed by connecting adjacent lands horizontally or vertically. You may assume all four edges of the grid are all surrounded by water.

Example 1:

```
Input: grid = [
  ["1","1","1","1","0"],
  ["1","1","0","1","0"],
  ["1","1","0","0","0"],
  ["0","0","0","0","0"]
]
Output: 1
```

Example 2:

```
Input: grid = [
  ["1","1","0","0","0"],
  ["1","1","0","0","0"],
  ["0","0","1","0","0"],
  ["0","0","0","1","1"]
]
Output: 3
```

grid =

	0	1	2	3	4
0	1	1	0	0	0
1	1	1	0	0	0
2	0	0	1	0	0
3	0	0	0	1	1

Output : 3

for row in num_rows:

for col in num_cols:

if (grid[row][col] == 1)

if (not vis[row][col])

bfs(row, col)

count += 1

208. Implement Trie (Prefix Tree)

Solved

Medium Topics Companies

A **trie** (pronounced as "try") or **prefix tree** is a tree data structure used to efficiently store and retrieve keys in a dataset of strings. There are various applications of this data structure, such as autocomplete and spellchecker.

Implement the Trie class:

- `Trie()` Initializes the trie object.
- `void insert(String word)` Inserts the string `word` into the trie.
- `boolean search(String word)` Returns `true` if the string `word` is in the trie (i.e., was inserted before), and `false` otherwise.
- `boolean startsWith(String prefix)` Returns `true` if there is a previously inserted string `word` that has the prefix `prefix`, and `false` otherwise.

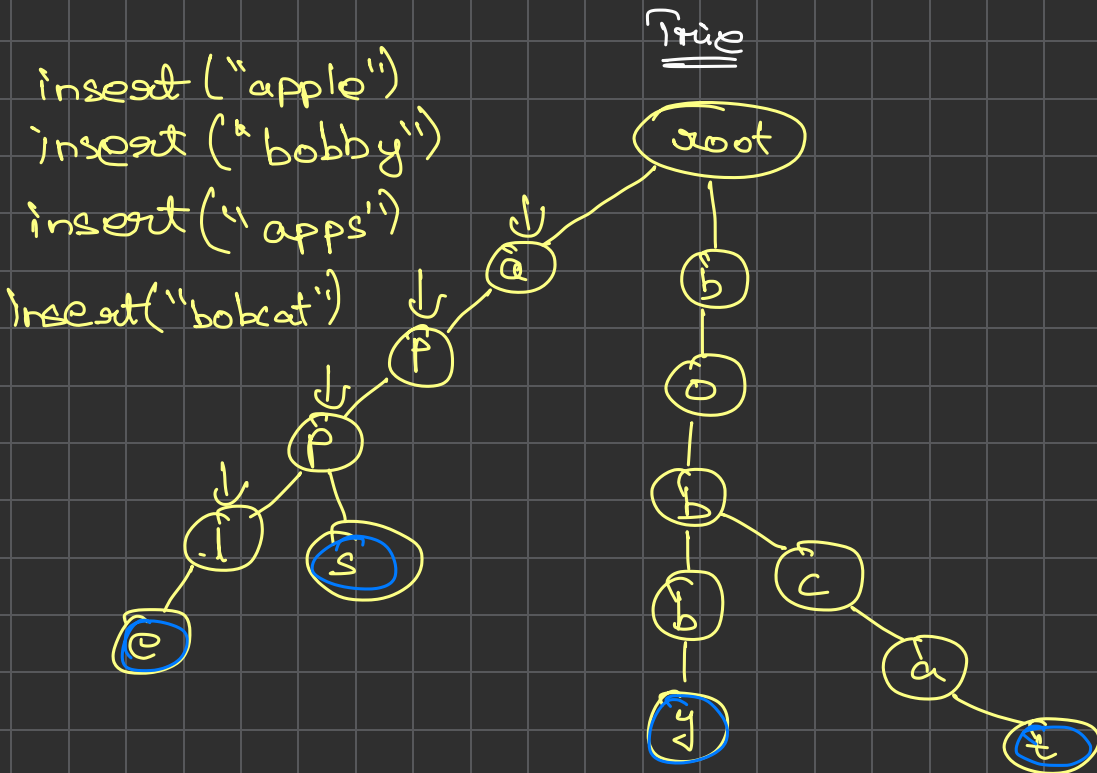
Example 1:

Input
["Trie", "insert", "search", "search", "startsWith", "insert", "search"]
[[], ["apple"], ["apple"], ["app"], ["app"], ["app"], ["app"]]

Output
[null, null, true, false, true, null, true]

Explanation

```
Trie trie = new Trie();
trie.insert("apple");
trie.search("apple"); // return True
trie.search("app"); // return False
trie.startsWith("app"); // return True
trie.insert("app");
trie.search("app"); // return True
```



$[\text{"apple"}] = \text{Trie}$

search("apple") → True
search("app") → False
startsWith("app") → True

TrieNode:

children:
end-of-word: bool

search("apple")

a p p l e

search("appl")
a p p l x

startsWith("appl")

insert(word): $O(n)$
search(word): $O(n)$
startsWith(prefix): $O(K)$; K } prefix size.

286. Walls and Gates Premium

Solved

Medium Topics Companies

You are given an $m \times n$ grid `rooms` initialized with these three possible values.

- `-1` A wall or an obstacle.
- `0` A gate.
- `INF` Infinity means an empty room. We use the value $2^{31} - 1 = 2147483647$ to represent `INF` as you may assume that the distance to a gate is less than 2147483647 .

Fill each empty room with the distance to *its nearest gate*. If it is impossible to reach a gate, it should be filled with `INF`.

Example 1:

3						↓3					↓1
2	2	1				↓2	2	1	↑		
1		2				↓1		2	↑		
								3	↑	4	

Input: `rooms = [[2147483647,-1,0,2147483647],[2147483647,2147483647,-1],[2147483647,-1,2147483647,-1],[0,-1,2147483647,2147483647]]`

Output: `[[3,-1,0,1],[2,2,1,-1],[1,-1,2,-1],[0,-1,3,4]]`

Route:

Apply DFS on each empty room
 $O(m \times n)^2$

Best:

$TC = O(m \times n)$

$S.C = O(m \times n)$